EXP 3	A PYTHON PROGRAMM TO IMPLEMENT THE ASSIGNMENT PROBLEM

AIM: To understand and solve the assignment problem using python, assignment team of workers.

PROCEDURE:

1. INPUT DATA: Define the cost matrix where each element represents the cost of assigning a particular task to a specific worker.

2. ALGORITHM: Use an optimization algorithm like the Hungarian algorithm (or the Munkres algorithm) to find the optimal assignment that minimizes the total cost.

3. OUTPUT: Display the optimal assignment along with the minimum total cost.

PROBLEM 1:

SOLVE THE ASSIGNMENT PROBLEM OF 4 JOBS AND 5 MACHINES FOR THE FOLLOWING MATRIX

	M1	M2	M3	M4	M5
J1	[12	7	9	11	8
J2	8	9	6	10	7
J3	7	14	12	8	9
J4	10	11	5	9	13]

CODE:

```
import numpy as np

from scipy.optimize import linear_sum_assignment

def solve_assignment(cost_matrix):

    original = cost_matrix.copy()
```

```
    rows, cols = cost_matrix.shape
    # Balance (add dummy row if needed)
    if rows < cols:
        dummy = np.zeros((cols - rows, cols))
        cost_matrix = np.vstack((cost_matrix, dummy))
    # Solve using Hungarian method
    r, c = linear_sum_assignment(cost_matrix)
    # Remove dummy assignments
    valid = r < original.shape[0]
    r, c = r[valid], c[valid]
    total_cost = original[r, c].sum()
    return r, c, total_cost
# New Cost Matrix
cost_matrix = np.array([
    [12, 7, 9, 11, 8],
    [8, 9, 6, 10, 7],
    [7, 14, 12, 8, 9],
    [10, 11, 5, 9, 13]
])
r, c, cost = solve_assignment(cost_matrix)
print("Optimal Assignment:")
for i in range(len(r)):
    print(f"Job {r[i]+1} -> Machine {c[i]+1}")
print("\nMinimum Cost:", cost)
```

OUTPUT:

```
Optimal Assignment:
Job 1 -> Machine 2
```

Job 2 -> Machine 3
Job 3 -> Machine 1
Job 4 -> Machine 4

Minimum Cost: 29

PROBLEM 2: Solve the assignment problem of 4 jobs and 5 machines for the following cost matrix:

	M1	M2	M3	M4	M5
J1	**9**	**7**	**8**	**6**	**9**
J2	**6**	**9**	**5**	**8**	**7**
J3	**5**	**8**	**9**	**7**	**6**

CODE:

```
import numpy as np
from scipy.optimize import linear_sum_assignment
def solve_assignment(cost_matrix):
    original = cost_matrix.copy()
    rows, cols = cost_matrix.shape
    if rows != cols:
        print("Unbalanced assignment problem detected.")
        # Add dummy rows
        if rows < cols:
            dummy = np.zeros((cols - rows, cols))
            cost_matrix = np.vstack((cost_matrix, dummy))
        # Add dummy columns
        elif cols < rows:
            dummy = np.zeros((rows, rows - cols))
            cost_matrix = np.hstack((cost_matrix, dummy))
```

```
        print("Padded Cost Matrix:")
        print(cost_matrix)
    # Solve
    r, c = linear_sum_assignment(cost_matrix)
    # Remove dummy assignments
    valid = (r < original.shape[0]) & (c < original.shape[1])
    r, c = r[valid], c[valid]
    total_cost = original[r, c].sum()
    return r, c, total_cost
# Given matrix (3x5)
cost_matrix = np.array([
    [9, 7, 8, 6, 9],
    [6, 9, 5, 8, 7],
    [5, 8, 9, 7, 6]
])
r, c, cost = solve_assignment(cost_matrix)
print("\nOptimal Assignment:")
for i in range(len(r)):
    print(f"Job {r[i]+1} -> Machine {c[i]+1}")

print("\nMinimum Cost:", cost)
```

OUTPUT:

```
Unbalanced assignment problem detected.
Padded Cost Matrix:
[[9. 7. 8. 6. 9.]
 [6. 9. 5. 8. 7.]
```

[5. 8. 9. 7. 6.]

[0. 0. 0. 0. 0.]

[0. 0. 0. 0. 0.]]

Optimal Assignment:

Job 1 -> Machine 4

Job 2 -> Machine 3

Job 3 -> Machine 1

Minimum Cost: 16

RESULT: Thus Understanding And Solving Of The Assignment Problem Using Python, Assignment Team Of Workers.